0.1 Continued fractions

The continued fraction

$$\cfrac{1}{\sqrt{2}+\cfrac{1}{\sqrt{2}+\cfrac{1}{\sqrt{2}+\cdots}}} \tag{1}$$

can be obtained by typing

```
\cfrac{1}{\sqrt{2}+
 \cfrac{1}{\sqrt{2}+
  \cfrac{1}{\sqrt{2}+\dotsb
}}}
```

This produces better-looking results than straightforward use of `\frac`. Left or right placement of any of the numerators is accomplished by using `\cfrac[l]` or `\cfrac[r]` instead of `\cfrac`.

0.2 Smash options

The command `\smash` is used to typeset a subformula with an effective height and depth of zero, which is sometimes useful in adjusting the subformula's position with respect to adjacent symbols. With the amsmath package `\smash` has optional arguments `[t]` and `[b]` because occasionally it is advantageous to be able to "smash" only the top or only the bottom of something while retaining the natural depth or height. For example, when adjacent radical symbols are unevenly sized or positioned because of differences in the height and depth of their contents, `\smash` can be employed to make them more consistent. Compare $\sqrt{x}+\sqrt{y}+\sqrt{z}$ and $\sqrt{x}+\sqrt{y}+\sqrt{z}$, where the latter was produced by `$\sqrt{x} + \sqrt{\smash[b]{y}} + \sqrt{z}$`.

0.3 Delimiters

0.3.1 Delimiter sizes

The automatic delimiter sizing done by `\left` and `\right` has two limitations: First, it is applied mechanically to produce delimiters large enough to encompass the largest contained item, and second, the range of sizes is not even approximately continuous but has fairly large quantum jumps. This means that a math fragment that is infinitesimally too large for a given delimiter size will get the next larger size, a jump of 3pt or so in normal-sized text. There are two or three situations where the delimiter size is commonly adjusted, using a set of commands that have 'big' in their names.

Delimiter size	text size	`\left` `\right`	`\bigl` `\bigr`	`\Bigl` `\Bigr`	`\biggl` `\biggr`	`\Biggl` `\Biggr`
Result	$(b)(\frac{c}{d})$	$(b)\left(\frac{c}{d}\right)$	$(b)\bigl(\frac{c}{d}\bigr)$	$\Bigl(b\Bigr)\Bigl(\frac{c}{d}\Bigr)$	$\biggl(b\biggr)\biggl(\frac{c}{d}\biggr)$	$\Biggl(b\Biggr)\Biggl(\frac{c}{d}\Biggr)$

The first kind of situation is a cumulative operator with limits above and below. With `\left` and `\right` the delimiters usually turn out larger than necessary, and using the `Big` or `bigg` sizes instead gives better results:

$$\left[\sum_i a_i \left|\sum_j x_{ij}\right|^p\right]^{1/p} \quad \text{versus} \quad \biggl[\sum_i a_i \Bigl|\sum_j x_{ij}\Bigr|^p\biggr]^{1/p}$$

```
\biggl[\sum_i a_i\Bigl\lvert\sum_j x_{ij}\Bigr\rvert^p\biggr]^{1/p}
```

The second kind of situation is clustered pairs of delimiters where `\left` and `\right` make them all the same size (because that is adequate to cover the encompassed material) but what you really want is to make some of the delimiters slightly larger to make the nesting easier to see.

$$\left((a_1b_1) - (a_2b_2)\right)\left((a_2b_1) + (a_1b_2)\right) \quad \text{versus} \quad \bigl((a_1b_1) - (a_2b_2)\bigr)\bigl((a_2b_1) + (a_1b_2)\bigr)$$

```
\left((a_1 b_1) - (a_2 b_2)\right)
\left((a_2 b_1) + (a_1 b_2)\right)
\quad\text{versus}\quad
\bigl((a_1 b_1) - (a_2 b_2)\bigr)
\bigl((a_2 b_1) + (a_1 b_2)\bigr)
```

The third kind of situation is a slightly oversize object in running text, such as $\left|\frac{b'}{d'}\right|$ where the delimiters produced by `\left` and `\right` cause too much line spreading. In that case `\bigl` and `\bigr` can be used to produce delimiters that are slightly larger than the base size but still able to fit within the normal line spacing: $\bigl|\frac{b'}{d'}\bigr|$.

In ordinary LaTeX `\big`, `\bigg`, `\Big`, and `\Bigg` delimiters aren't scaled properly over the full range of LaTeX font sizes. With the amsmath package they are.

0.3.2 Vertical bar notations

The amsmath package provides commands `\lvert`, `\rvert`, `\lVert`, `\rVert` (compare `\langle`, `\rangle`) to address the problem of overloading for the vert bar character `|`. This character is currently used in LaTeX documents to represent a wide variety of mathematical objects: the 'divides' relation in a number-theory expression like $p|q$, or the absolute-value operation $|z|$, or the 'such that' condition in set notation, or the 'evaluated at' notation $f_\zeta(t)\big|_{t=0}$. The multiplicity of uses in itself is not so bad; what is bad, however, is that fact that not all of the uses take the same typographical treatment, and that the complex discriminatory powers of a knowledgeable reader cannot be replicated in computer processing of mathematical documents. It is recommended therefore that there should be a one-to-one correspondence in any given document between the vert bar character `|` and a selected mathematical notation, and similarly for the double-bar command `\|`. This immediately rules out the use of `|` and `\|` for delimiters, because left and right delimiters are distinct usages

that do not relate in the same way to adjacent symbols; recommended practice is therefore to define suitable commands in the document preamble for any paired-delimiter use of vert bar symbols:

```
\providecommand{\abs}[1]{\lvert#1\rvert}
\providecommand{\norm}[1]{\lVert#1\rVert}
```

whereupon the document would contain `\abs{z}` to produce $|z|$ and `\norm{v}` to produce $\|v\|$. The `mathtools` provides the command `\DeclarePairedDelimiter` for defining `\abs`-like macros with scaling delimiters.

—1—

Operator names

1.1 Defining new operator names

Math functions such as log, sin, and lim are traditionally typeset in roman type to make them visually more distinct from one-letter math variables, which are set in math italic. The more common ones have predefined names, `\log`, `\sin`, `\lim`, and so forth, but new ones come up all the time in mathematical papers, so the amsmath package provides a general mechanism for defining new 'operator names'. To define a math function `\xxx` to work like `\sin`, you write

```
\DeclareMathOperator{\xxx}{xxx}
```

whereupon ensuing uses of `\xxx` will produce xxx in the proper font and automatically add proper spacing on either side when necessary, so that you get $A\,\mathrm{xxx}\,B$ instead of $A\mathrm{xxx}B$. In the second argument of `\DeclareMathOperator` (the name text), a pseudo-text mode prevails: the hyphen character `-` will print as a text hyphen rather than a minus sign and an asterisk `*` will print as a raised text asterisk instead of a centered math star. (Compare a-b^*c and $a - b * c$.) But otherwise the name text is printed in math mode, so that you can use, e.g., subscripts and superscripts there.

If the new operator should have subscripts and superscripts placed in 'limits' position above and below as with lim, sup, or max, use the `*` form of the `\DeclareMathOperator` command:

```
\DeclareMathOperator*{\Lim}{Lim}
```

See also the discussion of subscript placement in Section **??**.

The following operator names are predefined:

\arccos	arccos	\deg	deg	\lg	lg	\projlim	proj lim
\arcsin	arcsin	\det	det	\lim	lim	\sec	sec
\arctan	arctan	\dim	dim	\liminf	lim inf	\sin	sin
\arg	arg	\exp	exp	\limsup	lim sup	\sinh	sinh
\cos	cos	\gcd	gcd	\ln	ln	\sup	sup
\cosh	cosh	\hom	hom	\log	log	\tan	tan
\cot	cot	\inf	inf	\max	max	\tanh	tanh
\coth	coth	\injlim	inj lim	\min	min		
\csc	csc	\ker	ker	\Pr	Pr		

\varinjlim	$\varinjlim$	\varliminf	$\varliminf$
\varprojlim	$\varprojlim$	\varlimsup	$\varlimsup$

There is also a command \operatorname such that using

```
\operatorname{abc}
```

in a math formula is equivalent to a use of \abc defined by \DeclareMathOperator. This may be occasionally useful for constructing more complex notation or other purposes. (Use the variant \operatorname* to get limits.)

1.2 \mod and its relatives

Commands \mod, \bmod, \pmod, \pod are provided to deal with the special spacing conventions of "mod" notation. \bmod and \pmod are available in LaTeX, but with the amsmath package the spacing of \pmod will adjust to a smaller value if it's used in a non-display-mode formula. \mod and \pod are variants of \pmod preferred by some authors; \mod omits the parentheses, whereas \pod omits the "mod" and retains the parentheses.

(1.1) $\gcd(n, m \bmod n); \quad x \equiv y \pmod b; \quad x \equiv y \mod c; \quad x \equiv y \pod d$

```
\gcd(n,m\bmod n);\quad x\equiv y\pmod b;
\quad x\equiv y\mod c;\quad x\equiv y\pod d
```

—2—

The \text command

The main use of the command \text is for words or phrases in a display. It is very similar to the LaTeX command \mbox in its effects, but has a couple of advantages. If you want a word or phrase of text in a subscript, you can type `..._{\textrm{word or phrase}}`, which is slightly easier than the \mbox equivalent: `..._{\mbox{\rmfamily\scriptsize word or phrase}}`. Note that the standard \textrm command will use the amsmath \text definition, but ensure the \rmfamily font is used.

(2.1) $f_{[x_{i-1},x_i]} \text{ is monotonic,} \quad i = 1, \dots, c+1$

```
f_{[x_{i-1},x_i]} \text{ is monotonic,}
\quad i = 1,\dots,c+1
```

The font used for `\text` is the same as that of the surrounding environment; i.e., within a theorem, the contents of `\text` will be set in italic.

If a math expression is included in a `\text` string, it must be explicitly marked as math (`$...$`).

$$\partial_s f(x) = \frac{\partial}{\partial x_0} f(x) \quad \text{for } x = x_0 + I x_1.$$

```
\partial_s f(x) = \frac{\partial}{\partial x_0} f(x)\quad
  \text{for $x= x_0 + I x_1$.}
```

Function names should *not* be entered as `\text`. Instead, use `\mathrm` or `\DeclareMathOperator` as appropriate. These are fixed entities that should not change depending on outside content (such as appearing within a theorem that is set in italic), and in the case of declared operators, proper spacing is applied automatically.